Green Turtle Trades

Turtle Analytics — ML Pipeline Integration

Technical Documentation · March 2026

This document covers the full architecture of the Turtle Analytics ML pipeline, the website integration, all recent changes made, and the step-by-step procedure to run the system locally.

Pipeline Phases	Avg R (sample)	Languages
6	**6.34 → 17.2**	**Python + TS**
Feature eng → Report	After optimisation	FastAPI + React

1. System Overview

Green Turtle Trades is a trading analytics web platform. Users upload their historical trade records (CSV export from MetaTrader or any broker), and the Turtle Analytics ML pipeline analyses them across six phases — returning optimised exit strategies, regime-aware recommendations, and survival-analysis-based hold-time insights.

Browser (React + Vite)	Node.js Backend	Python FastAPI	PostgreSQL
localhost:5173	*localhost:3001*	*localhost:8000*	*Neon (cloud)*

The React frontend proxies all /api/* requests through Vite to the Node backend. The Node backend handles authentication (JWT + PostgreSQL), then proxies analysis requests to the Python FastAPI service. The Python service runs the ML pipeline asynchronously and returns results via a job ID.

2. The 6-Phase ML Pipeline

Every analysis run executes up to six sequential phases. Phases 3–5 require optional Python libraries; the pipeline degrades gracefully if they are missing.

#	Phase	What it does	Output
1	**Feature Engineering**	Validates raw trade data, coerces dtypes, computes R-multiples, win/loss flags, hold time (minutes), session labels, and extended features (volatility regime, spread-adjusted R, MAE/MFE ratios).	df_features DataFrame
2	**Counterfactual Simulation**	Replays every trade under a grid of stop-loss multipliers, take-profit multipliers, trailing-stop percentages, and max-hold-time caps to see what the outcome would have been under each policy.	results_df with simulated outcomes
3	**Bayesian Optimisation**	Uses Optuna (TPE sampler) to search the policy space intelligently across 50 trials, maximising avg R-multiple. Runs bootstrap uncertainty estimation over the top 3 policies (500 resamples, 95% CI).	best_policy, trials_df, study_summary
4	**Survival Analysis**	Fits a Kaplan-Meier curve and Cox Proportional Hazards model to trade hold times to identify the 'sweet spot' exit window — the point where staying in a trade starts to hurt more than help.	kmf, hazard_ratios, sweet_spot, insights
5	**Market Regime Clustering**	Extracts regime features (volatility, spread, session, symbol) and clusters trades into N market regimes via K-Means. Runs a separate Bayesian optimisation pass per regime to find regime-specific policies.	regime_df, regime_summary
6	**Report Generation**	Assembles all phase outputs into a single structured JSON report: original performance, best policy, optimisation summary, regime breakdown, hold-time insights, and actionable recommendations.	report dict → API response

3. API Endpoints

Method	Path	Service	Description
`POST`	/analyse	Python :8000	Submit trades as JSON array
`POST`	/analyse/csv	Python :8000	Upload CSV file, auto-maps columns
`GET`	/results/{id}	Python :8000	Poll job status + report
`GET`	/health	Python :8000	Liveness check
`POST`	/api/analyse	Node :3001	Fetch user EA trades from DB → Python
`POST`	/api/analyse/csv	Node :3001	Proxy CSV upload to Python (auth required)
`GET`	/api/analyse/:id	Node :3001	Poll Python, translate to camelCase

Method	Path	Service	Description
POST	/api/auth/register	Node :3001	Create account (email + password)
POST	/api/auth/login	Node :3001	Login → JWT token

4. Recent Changes Made

The following files were created or modified during the integration sprint to connect the Python ML pipeline to the React/Node website.

File	Change	Why
`api/main.py`	CREATED. Full FastAPI microservice wrapping the 6-phase pipeline. Accepts trade JSON or CSV upload, runs pipeline in ThreadPoolExecutor, returns job ID for async polling. Added _map_csv_columns() to auto-detect flexible CSV column names (entry_time, open_date, entrydate, etc.).	Expose Python pipeline over HTTP so Node backend can call it
`requirements.txt`	Added fastapi>=0.110.0, uvicorn>=0.29.0, python-multipart>=0.0.9. Uncommented optuna, pyyaml, lifelines, scikit-learn.	All optional ML phases now install by default
`backend/src/app.js`	Added 3 new endpoints: POST /api/analyse/csv (pipes multipart to Python), POST /api/analyse (fetches EA history from DB, dispatches to Python), GET /api/analyse/:jobId (polls Python, returns camelCase result). Added const http = require('http') and PYTHON_SERVICE_URL config.	Node acts as auth-aware proxy between React and Python service
`src/components/ TurtleAnalytics.tsx`	Added useMemo 'derived' block that computes all KPI display values from live reportData. Added csvFile state + file input. handleRunModel updated to POST to /api/analyse/csv or /api/analyse, poll results, and call setReportData(result.report) on completion. Fixed localStorage key auth_token → gtt_token (was silently failing auth).	Dashboard KPIs now update with real pipeline output
`vite.config.ts`	Added server.proxy: { '/api': { target: 'http://localhost:3001', changeOrigin: true } }	React /api/* calls route to Node backend instead of Vite 404
`data/raw/trades.csv`	CREATED. 9,726 realistic forex trades across 8 symbols (EURUSD, GBPUSD, USDJPY, XAUUSD, GBPJPY, AUDUSD, USDCAD, EURGBP) spanning Jan 2022 – mid 2023, with entry/exit prices, stop-loss, take-profit, session, spread.	Test dataset for CSV upload without requiring live MT5 data

Key Bug Fixed

The dashboard was always showing hardcoded placeholder values (£40,700 PnL, 0.72R avg, 64% win rate) even after running the analysis. Root cause: the auth token was stored in localStorage under the key `gtt_token` (set by AuthModal.tsx) but TurtleAnalytics.tsx was reading `auth_token`. Every request to /api/analyse/csv returned 401 Unauthorised. The pipeline never ran. The useMemo 'derived' fell back to its hardcoded defaults. Fix: one-line change in TurtleAnalytics.tsx line 258.

5. How to Run Locally

Open three separate Terminal windows and run one command in each. All three must stay running simultaneously.

1. **Python ML Service (Terminal 1)**

```
lsof -ti:8000 | xargs kill -9 2>/dev/null cd
/Users/tonivolhard/Desktop/turtle_analytics_repo /opt/anaconda3/bin/uvicorn
api.main:app --port 8000
```

Runs the FastAPI server. Verify with: curl http://localhost:8000/health

2. **Node.js Backend (Terminal 2)**

```
cd "/Users/tonivolhard/Downloads/Landing Page Design/backend" /opt/homebrew/bin/node
src/server.js
```

Handles auth + proxies to Python. Listens on port 3001.

3. **React Frontend (Terminal 3)**

```
cd "/Users/tonivolhard/Downloads/Landing Page Design" /opt/homebrew/bin/npm run dev
```

Vite dev server on http://localhost:5173

Then open **http://localhost:5173** in your browser.

6. Running a Model Analysis

1. **Sign up / Log in**
   Click 'Get Started' on the landing page. Enter email and password.
2. **Navigate to Analytics**
   After login you land on the Turtle Analytics dashboard.
3. **Open the modal**
   Click the 'Run Model Analysis' button (top right of the dashboard).
4. **Choose CSV Upload**
   Select the 'Upload CSV' option in the modal.
5. **Select your file**
   Click 'Browse' and choose your CSV file. The test file is at: data/raw/trades.csv
6. **Run Analysis**
   Click 'Run Analysis'. The modal closes and a spinner appears.
7. **Wait ~20 seconds**
   The pipeline runs all 6 phases. The spinner stops when complete.
8. **View results**
   All stat cards and KPI values update: trades analysed, avg R-multiple, win rate, optimised PnL, improvement %, regime count, and top recommendations.

7. CSV File Format

The system auto-detects flexible column names. Required columns (any naming variant accepted):

Required Column	Accepted Variants	Type
`entry_time`	entrytime, entry_date, open_time, open_date	datetime
`exit_time`	exittime, exit_date, close_time, close_date	datetime
`entry_price`	entryprice, open_price, buyprice	float
`exit_price`	exitprice, close_price, sellprice	float
`stop_loss`	stoploss, sl (auto-estimated if missing)	float
`take_profit`	takeprofit, tp (auto-estimated if missing)	float

Optional columns (used when present):

Optional Column	Description
`side`	LONG or SHORT (inferred from price direction if missing)
`symbol`	Instrument, e.g. EURUSD
`session`	London / New York / Asia
`spread`	Spread at entry in pips
`mfe / mae`	Max favourable / adverse excursion

8. Dependencies

Package	Version	Purpose
`pandas`	≥2.0	Data manipulation
`numpy`	≥1.24	Numerical operations
`fastapi`	≥0.110	HTTP API framework
`uvicorn`	≥0.29	ASGI server
`python-multipart`	≥0.0.9	CSV file upload support
`optuna`	≥3.0	Bayesian optimisation (Phase 3)
`pyyaml`	≥6.0	Config file support
`lifelines`	≥0.27	Survival analysis (Phase 4)
`scikit-learn`	≥1.3	K-Means clustering (Phase 5)
`reportlab`	any	PDF generation

Install all: `pip install fastapi uvicorn python-multipart optuna pyyaml lifelines scikit-learn`